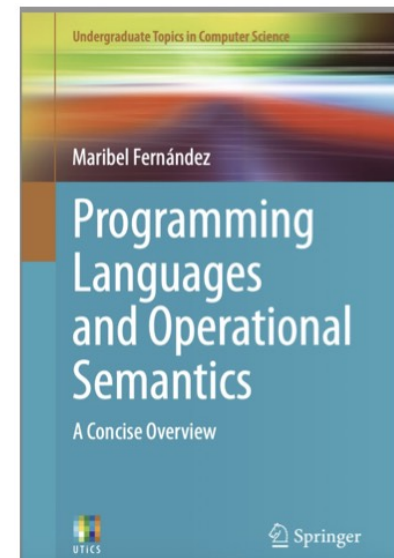
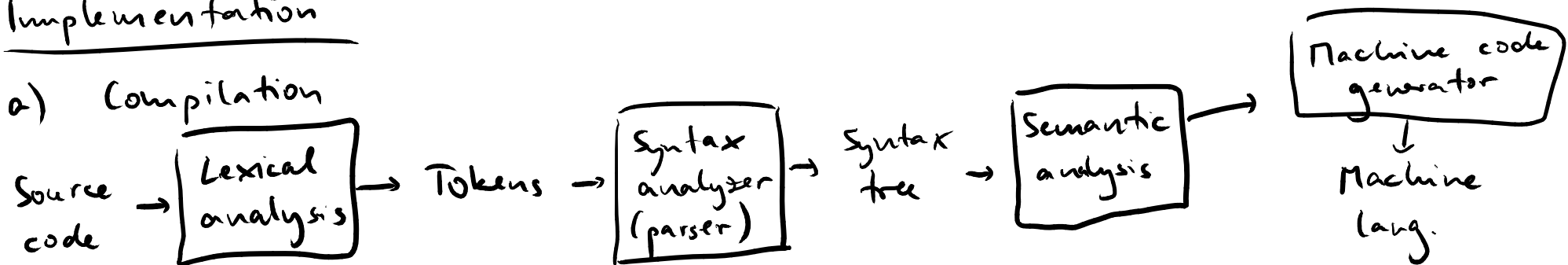


$PL = \underset{\substack{\vdots \\ \text{form}}}{\text{syntax}} + \underset{\substack{\vdots \\ \text{meaning}}}{\text{semantics}} + \underset{\substack{\vdots \\ \text{execution}}}{\text{implementation}}$



Implementation

a) Compilation



b) Interpretation

c) Hybrid

↳ e.g., Java, JavaScript

Syntax

a) Grammar $\rightarrow$ Which programs are syntactically correct?

4 parts: terminals Σ , non-terminals N , productions P ,
initial symbol $s \in N$

Example: Arithmetic expression

$$\Sigma = \{0, 1, 2, \dots, 9, +, -\}$$

$$N = \{\text{Exp}, \text{Num}, \text{Op}, \text{Digit}\}$$

$$s = \text{Exp}$$

$$P = \text{Exp} \rightarrow \text{Num} \mid \text{Exp Op Exp}$$

$$\text{Op} \rightarrow + \mid -$$

$$\text{Num} \rightarrow \text{Digit} \mid \text{Digit Num}$$

$$\text{Digit} \rightarrow 0 \mid 1 \mid \dots \mid 9$$

What is part of the language?

A) $12 - 2$ ✓

B) $2 + (12 - 4)$ ✗

C) $11 * 4$ ✗

D) 1235470987 ✓

b) Abstract syntax trees

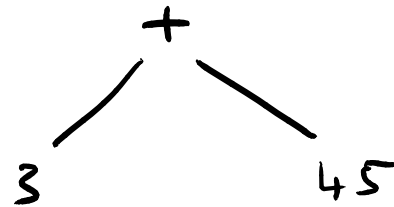
- abstract grammar

$$\hookrightarrow \text{e.g.}, \quad E \rightarrow n \mid \text{Op} (E, E)$$

$$\text{Op} \rightarrow + \mid -$$

- terminals = tokens

Example: $3 + 45$



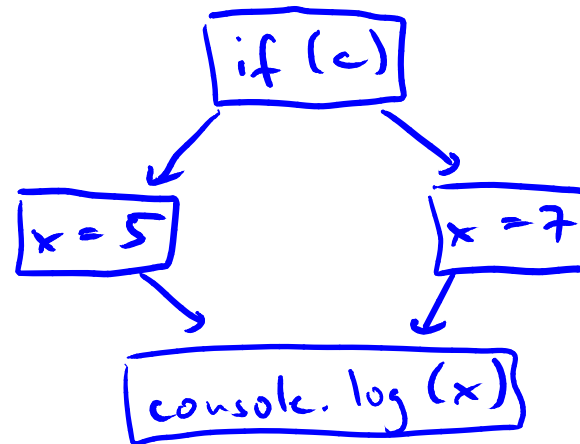
Control flow graphs

↳ Models flow of control through program

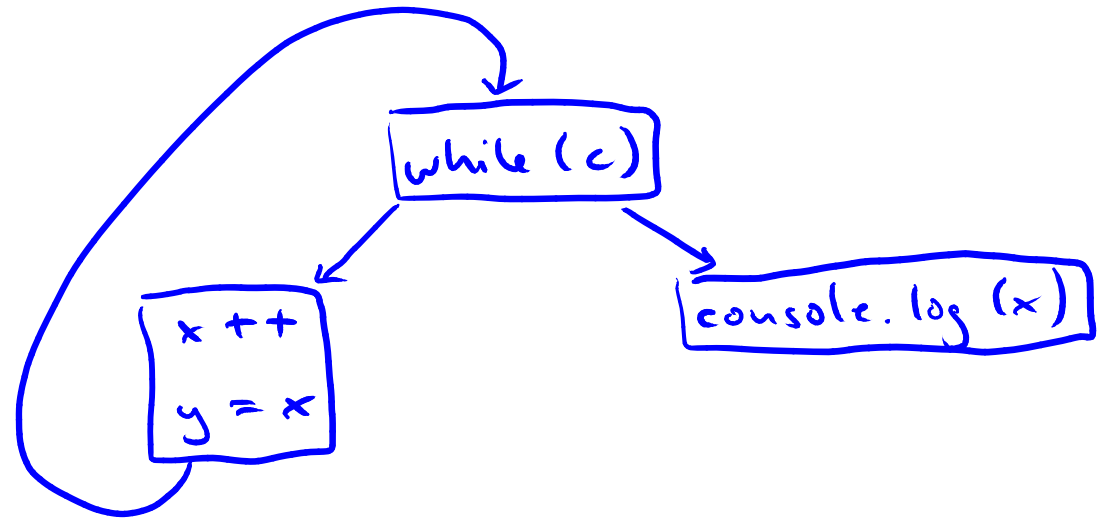
$G = (N, E)$ where N .. basic blocks (sequ. of operations executed together)

E .. possible transfer of control

Ex. 1 if (c)
 $x = 5$
 else
 $x = 7$
 console.log (x)



Ex. 2 while (c) {
 x++;
 y = x;
 }
 console.log(x);



Data Dependence Graphs

↳ Model flow of data from "definitions" to "uses"

$G = (N, E)$ where N .. operations

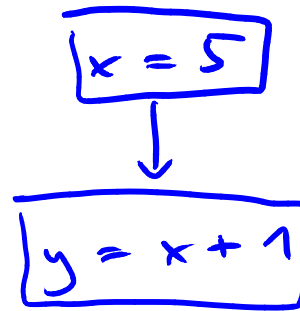
E .. possible def-use relations

$e = (n_1, n_2)$ means: n_2 may use data defined at n_1

Ex. 1

$x = 5$

$y = x + 1$



Ex. 2

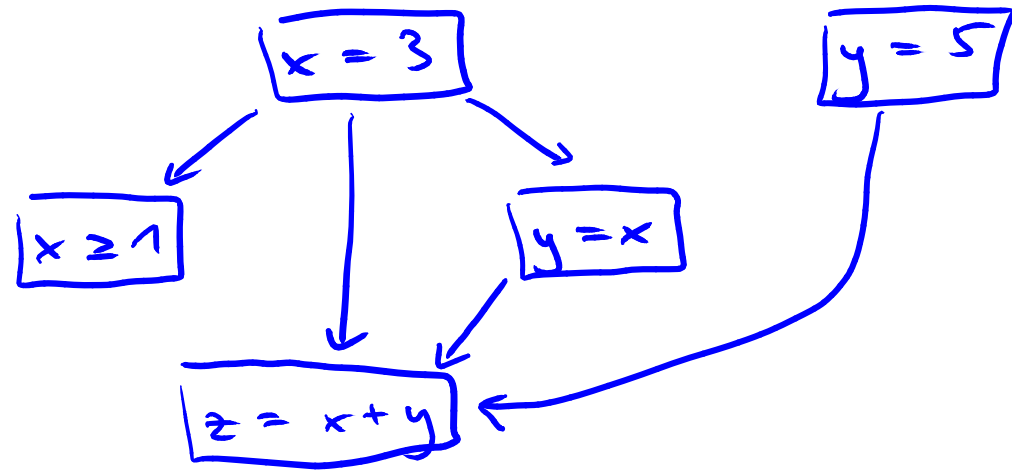
$x = 3$

$y = 5$

if ($x \geq 1$)

$y = x$

$z = x + y$



[Program Verification]

The Aim of Program Verification

* We want to provide guarantees about the behaviour of models and programs.

1. Correctness: Is the program correct? that is, does it have the intended functionality ?
2. Safety and Security: Does the program give the required safety and security guarantees?
3. Performance: Does the program live up to the required performance criteria?

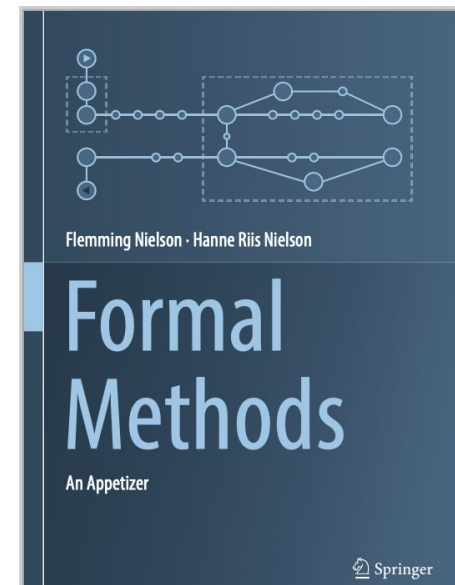
* What is needed to reason about such properties?

—> We need a precise understanding of the meaning of programs.

*This is provided by the semantics!

—> We need a precise description of the property of interest.

*For this we shall introduce predicates.



[Four Techniques for Reasoning about Properties]

1. Program Verification:

We attach general predicates to the node of the program graph and formally prove that they hold whenever control reaches the nodes.

2. Program Analysis:

We approximate the properties holding at the nodes. Compared to program verification, program analysis is less expressive but then it is fully automatic.

3. Language-based Security:

We restrict the attention to two specific properties of information flow, namely confidentiality and integrity . The resulting analyses are fully automatic and can be used to reject programs violating the security properties.

4. Model Checking: We express the properties in a restricted logic – and obtain a fully automatic technique. Model checking is more restrictive than program verification and more precise than program analysis ; however, it is also more costly.

[predicate]: A predicate is a logical expression used to specify the correctness of a program at specific points in its execution.

[program_graph]: A program graph is a graphical representation of the control structure of a program or system.

—> It is a directed graph that focuses on how a system proceeds from one point to the next by performing specific actions.

DEFINITION 1.2: A *program graph* **PG** consists of the following:

- **Q**: a finite set of *nodes*
- $q_{\triangleright}, q_{\blacktriangleleft} \in \mathbf{Q}$: two nodes called the *initial node* and the *final node*, respectively
- **Act**: a set of *actions*
- $\mathbf{E} \subseteq \mathbf{Q} \times \mathbf{Act} \times \mathbf{Q}$: a finite set of *edges*

An edge (q_o, α, q_*) has *source* node q_o and *target* node q_* , and it is labelled with the *action* α .

We shall *require* that the initial and final nodes are distinct and that there are no edges with source $q_{\blacktriangleleft}$.

We sometimes write $\mathbf{PG} = (\mathbf{Q}, q_{\triangleright} q_{\blacktriangleleft}, \mathbf{Act}, \mathbf{E})$ to summarise the components of the program graph.

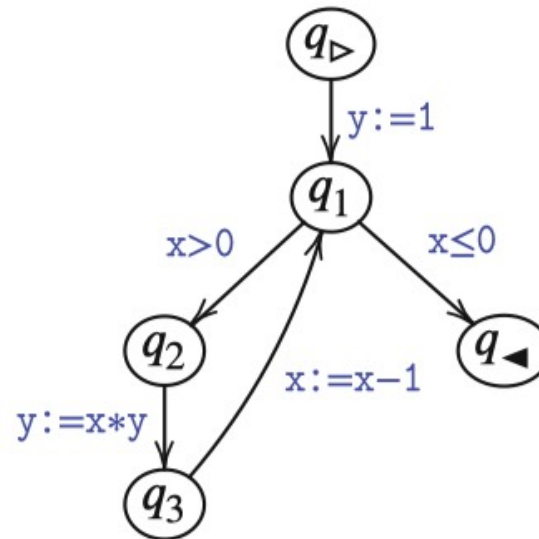


Figure 1.1: Program graph for the factorial function.

EXAMPLE 1.3: Returning to the program graph of Figure 1.1, it can be represented by the three sets

$$\mathbf{Q} = \{q_{\triangleright}, q_1, q_2, q_3, q_{\blacktriangleleft}\}$$

$$\mathbf{Act} = \{y := 1, x > 0, x \leq 0, y := x * y, x := x - 1\}$$

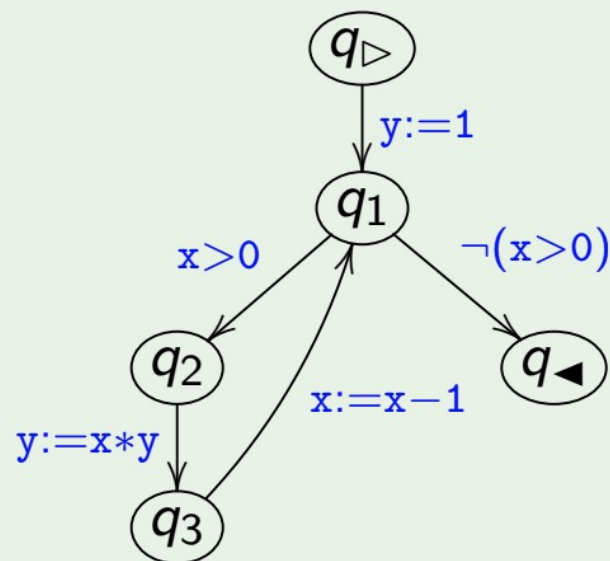
$$\mathbf{E} = \{(q_{\triangleright}, y := 1, q_1), (q_1, x > 0, q_2), (q_1, x \leq 0, q_{\blacktriangleleft}), \\ (q_2, y := x * y, q_3), (q_3, x := x - 1, q_1)\}$$

where the first set lists the nodes, including the initial and final nodes, the second set lists the actions, and the third set lists the edges.

Example: Factorial Function

How can we express that the program computes the factorial function?

Program Graph



Predicates (FM p 31)

We associate predicates with the nodes:

- $q_{\triangleright}: x = \underline{n} \geq 0$
- $q_{\blacktriangleleft}: y = \text{fac}(\underline{n})$

Here $\underline{n}$ is a variable not used (and hence not modified) in the program – it is called a *logical* variable.

Mathematical Function

$$\text{fac}(z) = \begin{cases} 1 & \text{if } z \leq 0 \\ z \cdot \text{fac}(z - 1) & \text{if } z > 0 \end{cases}$$